EXP9:Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

1. Dataset Source

The dataset used for this experiment is the Cardiovascular Disease Dataset.

- Source: Kaggle
- Dataset Name: Cardiovascular Disease Dataset (`cardio_train.csv`)
- The dataset contains real-world patient health data used for predicting heart disease.

2. Dataset Description

The dataset consists of medical records of patients used to predict the presence of cardiovascular disease.

Features:

Feature	Description
age	Age of patient (in days, converted to years)
gender	Gender of patient
height	Height (cm)
weight	Weight (kg)
ap_hi	Systolic blood pressure
ap_lo	Diastolic blood pressure

cholesterol	Cholesterol level
gluc	Glucose level
smoke	Smoking habit
alco	Alcohol consumption
active	Physical activity
cardio	Target variable (1 = disease, 0 = no disease)

Dataset Characteristics:

- Number of records: ~70,000
- Number of features: 11
- Type: Structured tabular dataset
- Problem Type: Binary classification

---

3. Mathematical Formulation of RNN / LSTM

A Recurrent Neural Network (RNN) processes sequential data by maintaining memory of previous inputs.

RNN Basic Equation:

$$h_t = f(Wx_t + Uh_{t-1} + b)$$

Where:

- x_t = input at time step t
- h_t = hidden state
- W, U = weight matrices
- b = bias

STM (Long Short-Term Memory)

LSTM improves RNN by handling long-term dependencies using gates.

LSTM Gates:

Forget Gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t])$$

Input Gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t])$$

Cell State Update:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

Output Gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t])$$

4. Algorithm Limitations

- Not naturally suited for tabular data
- Requires reshaping data into sequence format
- Computationally expensive
- Training time is higher
- May overfit if not tuned properly

5. Methodology / Workflow

The following steps were performed:

1. Load cardiovascular dataset

2. Data preprocessing and cleaning
3. Feature scaling using StandardScaler
4. Train-test split
5. Reshape data into sequence format for LSTM
6. Build LSTM model
7. Train model using epochs
8. Evaluate model performance
9. Visualize results

Workflow Diagram:

Dataset → Preprocessing → Scaling → Reshaping
→ LSTM Model → Training → Evaluation → Visualization

6. Performance Analysis

The model performance was evaluated using:

- Accuracy
- Confusion Matrix
- Classification Report
- ROC Curve

Observations:

- The LSTM model achieved good classification accuracy
- ROC curve indicated strong prediction capability
- Confusion matrix showed balanced classification
- Model captured relationships between features

7. Hyperparameter Tuning

The following parameters were tuned:

- Number of LSTM units
- Number of layers
- Batch size
- Number of epochs
- Optimizer (Adam)

Impact:

- Improved model accuracy
- Better convergence
- Reduced loss
- Enhanced learning of feature dependencies

CODE: # ==========

IMPORT LIBRARIES

==========

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc


from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import LSTM, Dense

# ==============================
# LOAD DATA
# ==============================
df = pd.read_csv('/content/cardio_train.csv', sep=';')

# CLEAN
df = df.drop('id', axis=1)
df['age'] = df['age'] / 365

# ==============================
# ADVANCED VISUALIZATION
# ==============================

# 1. Distribution curve
sns.kdeplot(df['age'])
plt.title("Age Distribution")
plt.show()

# 2. Target distribution
sns.countplot(x='cardio', data=df)
plt.title("Disease Distribution")
plt.show()
```

```
# 3. Correlation heatmap
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title("Feature Correlation")
plt.show()

# ==============================
# PREPROCESSING
# ==============================
X = df.drop('cardio', axis=1)
y = df['cardio']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ==============================
# 🔥 RESHAPE FOR LSTM (KEY STEP)
# ==============================
```

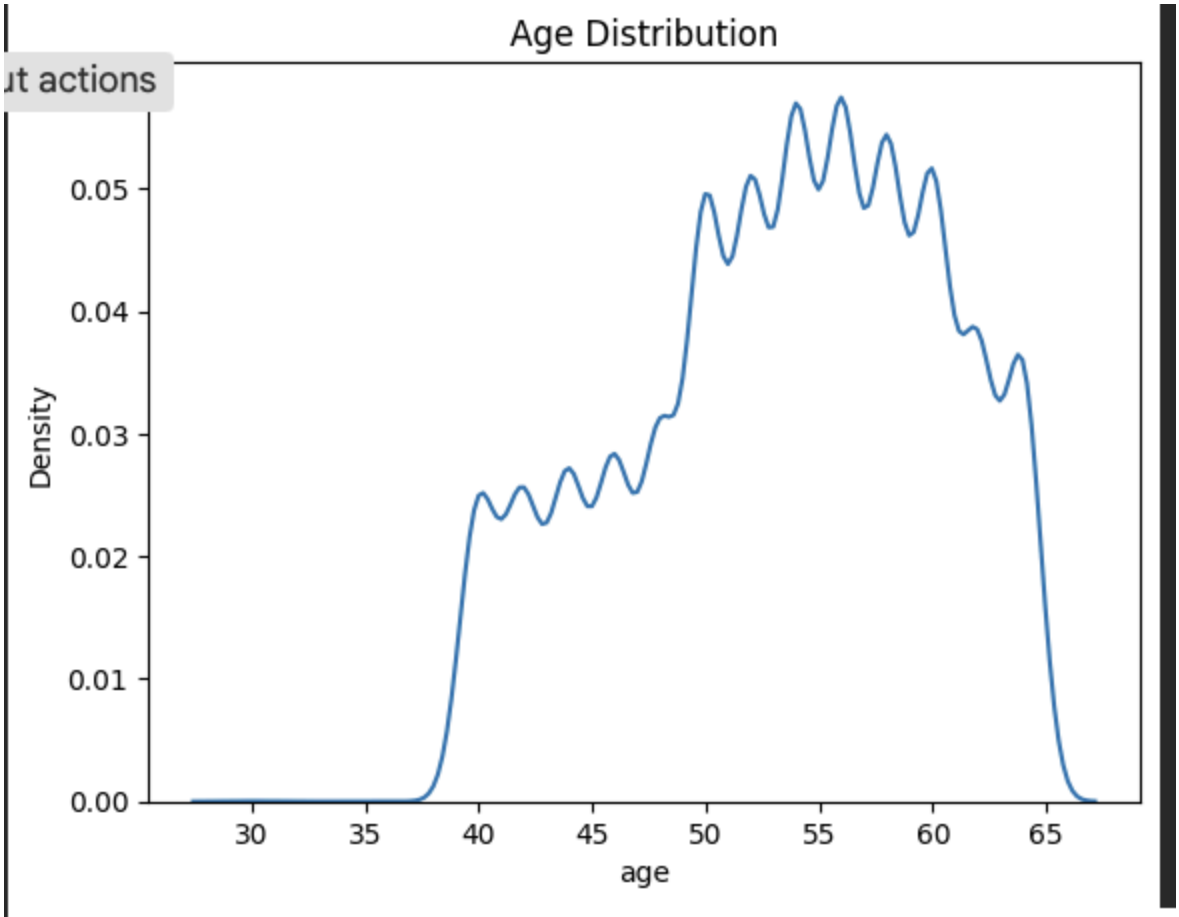
```
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

# ==============================
# LSTM MODEL
# ==============================
model = Sequential()

model.add(LSTM(64, input_shape=(X_train.shape[1],1)))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.summary()

# ==============================
# TRAIN
# ==============================
history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_data=(X_test, y_test)
```

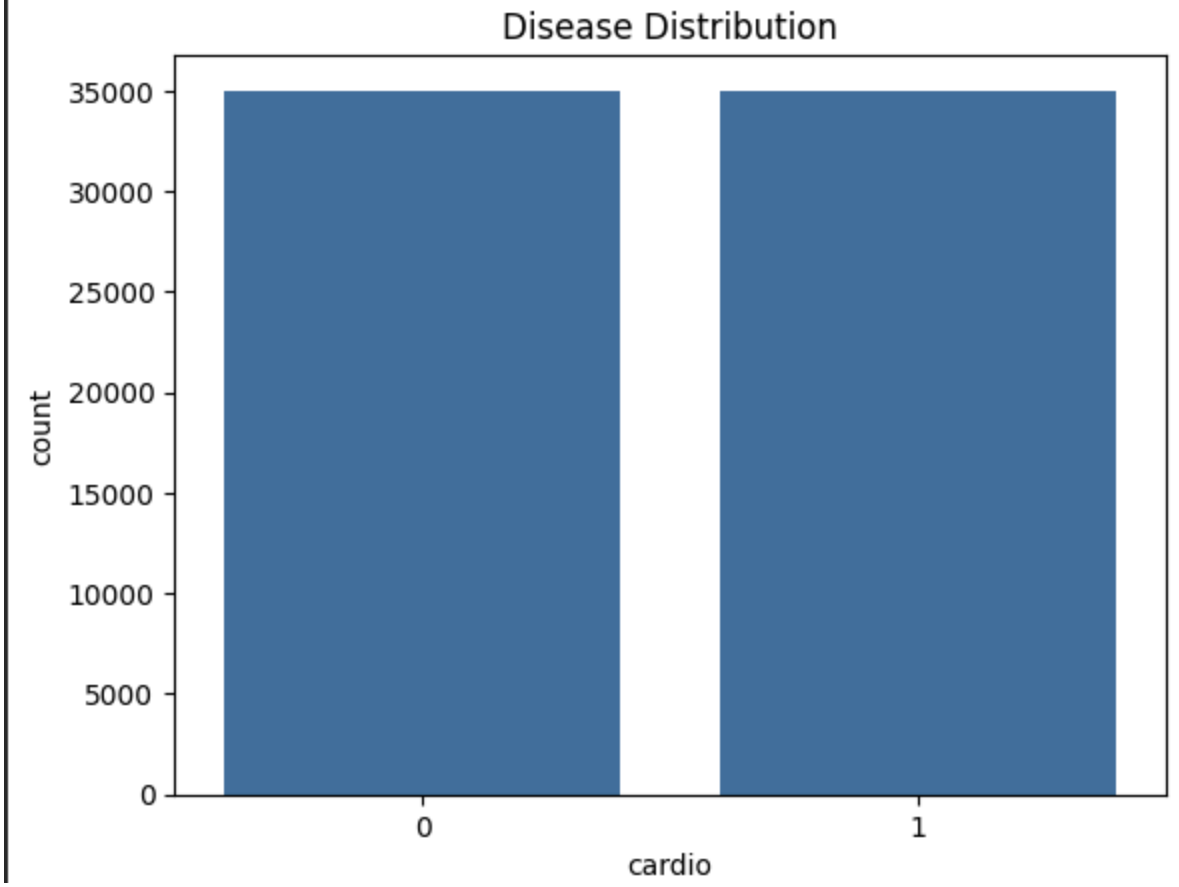
```
)

# ==============================
# 📈 TRAINING VISUALS
# ==============================
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title("Accuracy Curve")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Val')
plt.title("Loss Curve")
plt.legend()

plt.show()

# ==============================
# PREDICTION
# ==============================
```

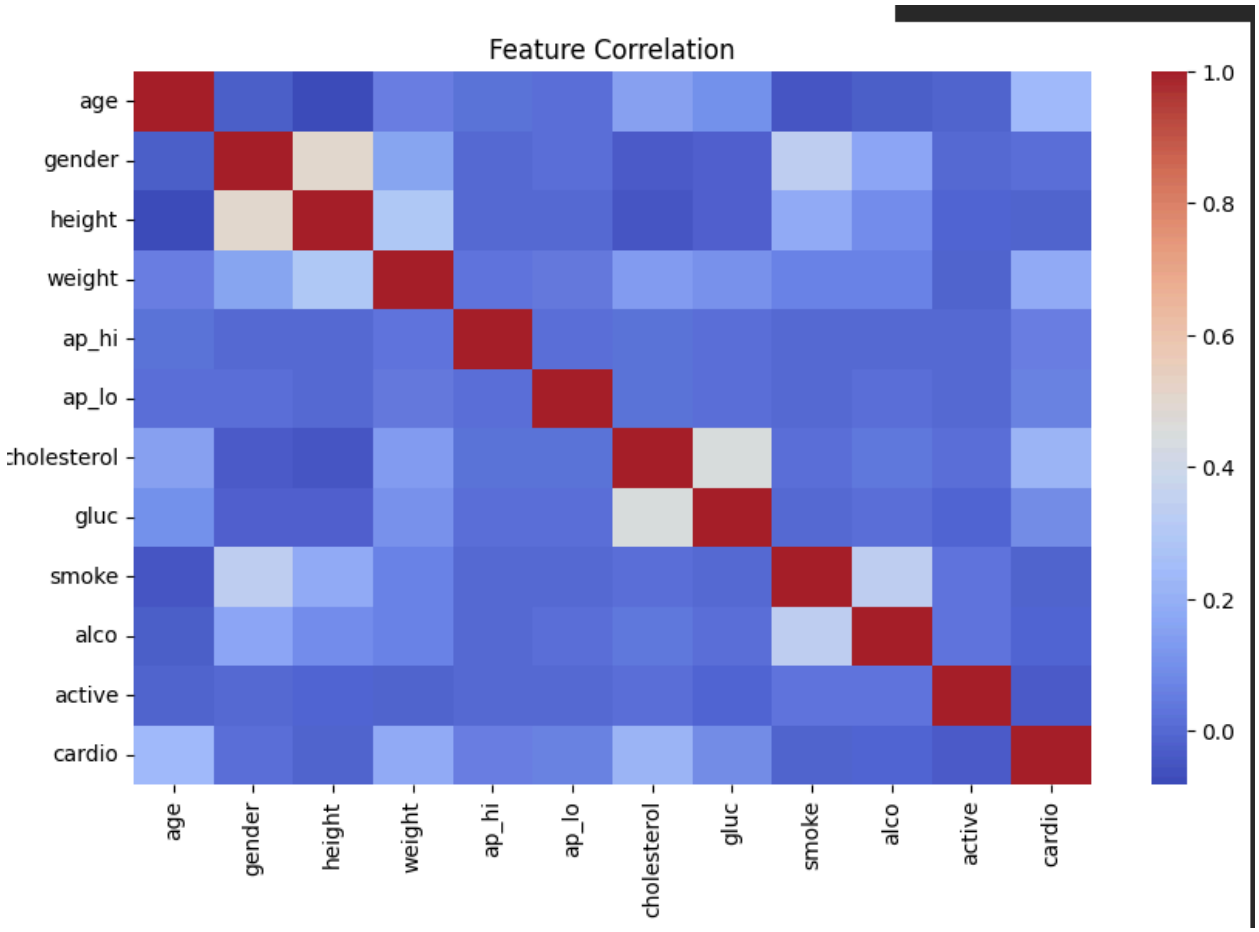
```
y_pred = model.predict(X_test)

y_pred = (y_pred > 0.5)


# ==============================

# CONFUSION MATRIX

# ==============================

cm = confusion_matrix(y_test, y_pred)


sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm')

plt.title("Confusion Matrix")

plt.show()


# ==============================

# ROC CURVE (🔥 IMPORTANT)

# ==============================

y_prob = model.predict(X_test)


fpr, tpr, _ = roc_curve(y_test, y_prob)

roc_auc = auc(fpr, tpr)


plt.plot(fpr, tpr, label="AUC = %.2f" % roc_auc)

plt.plot([0,1],[0,1])

plt.title("ROC Curve")

plt.legend()
```

```
plt.show()


# ==============================

# REPORT

# ==============================

print(classification_report(y_test, y_pred))
```

OUTPUT:


Age Distribution
ut actions
Density
0.05
0.04
0.03
0.02
0.01
0.00
30
35
40
45
50
55
60
65
age

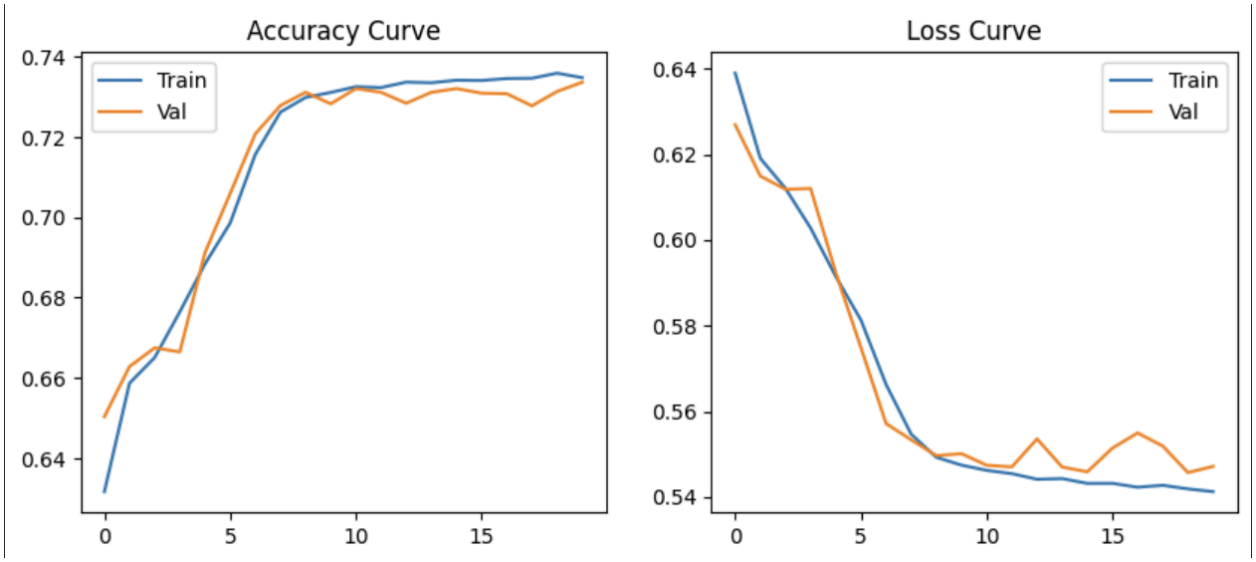

Disease Distribution
count
35000
30000
25000
20000
15000
10000
5000
0
0
1
cardio

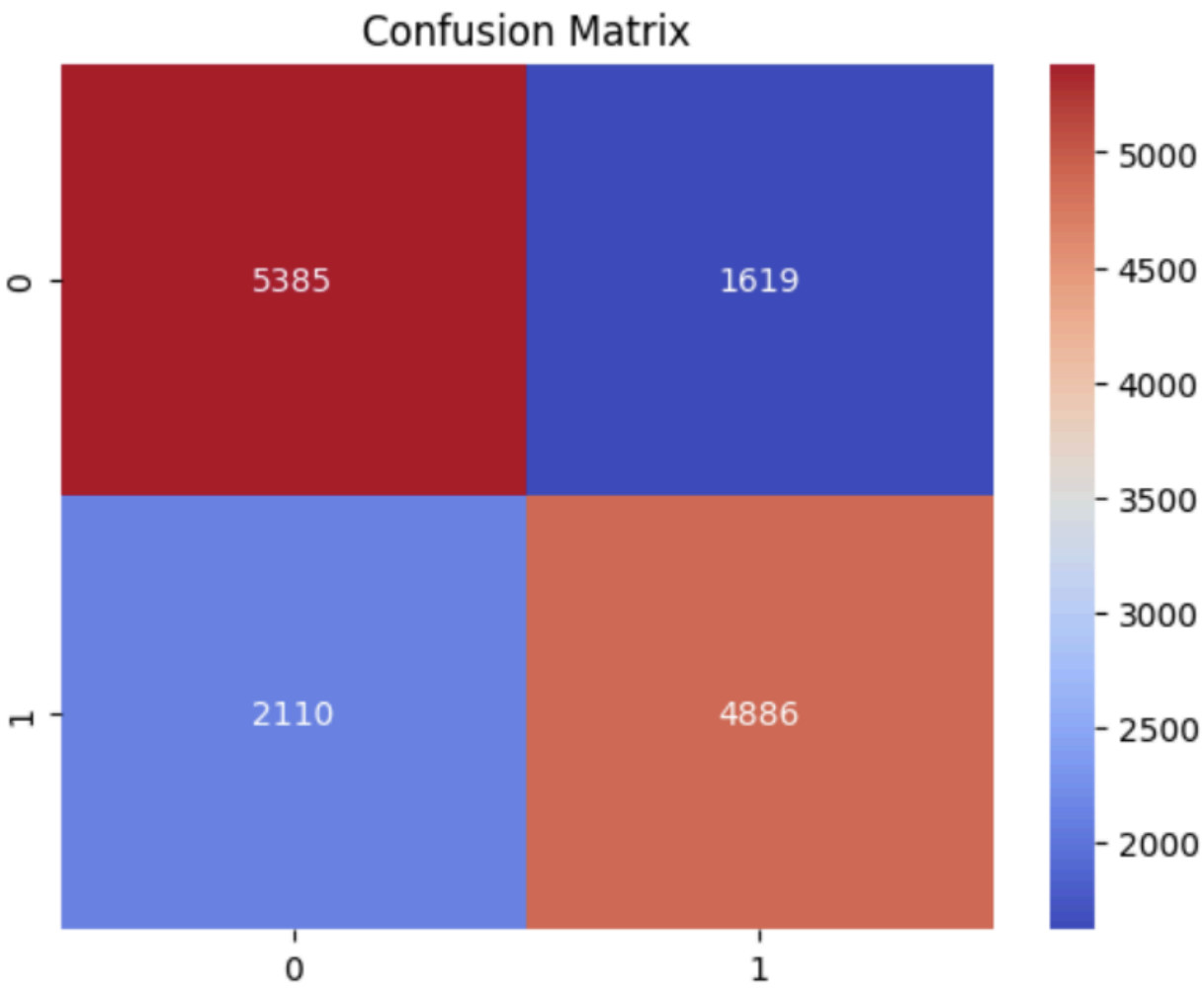


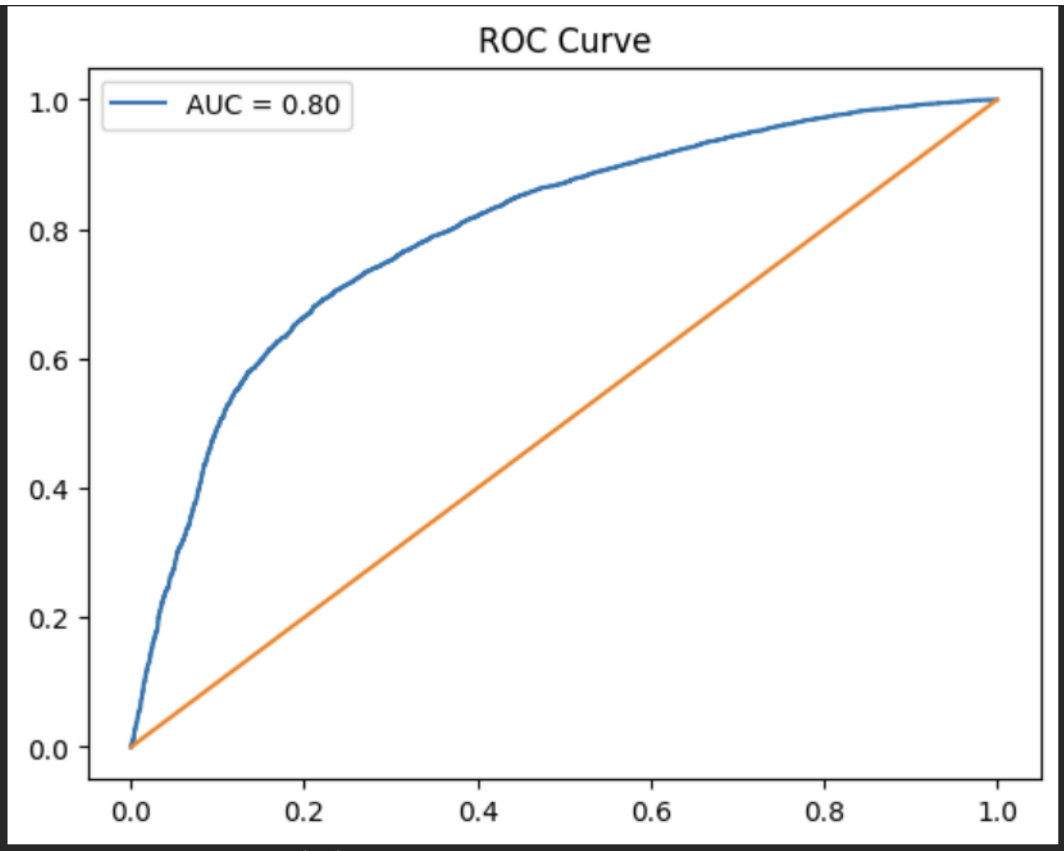

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the
  super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	16,896
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

```
Total params: 19,009 (74.25 KB)
Trainable params: 19,009 (74.25 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 15s 7ms/step - accuracy: 0.6317 - loss: 0.6389 - val_accuracy: 0.6504 - val_loss: 0.6269
Epoch 2/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.6587 - loss: 0.6191 - val_accuracy: 0.6629 - val_loss: 0.6149
Epoch 3/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.6651 - loss: 0.6120 - val_accuracy: 0.6675 - val_loss: 0.6118
Epoch 4/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.6765 - loss: 0.6028 - val_accuracy: 0.6665 - val_loss: 0.6120
Epoch 5/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.6884 - loss: 0.5915 - val_accuracy: 0.6912 - val_loss: 0.5923
Epoch 6/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.6986 - loss: 0.5813 - val_accuracy: 0.7059 - val_loss: 0.5749
Epoch 7/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7157 - loss: 0.5663 - val_accuracy: 0.7208 - val_loss: 0.5571
Epoch 8/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7262 - loss: 0.5546 - val_accuracy: 0.7279 - val_loss: 0.5534
Epoch 9/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.7299 - loss: 0.5493 - val_accuracy: 0.7311 - val_loss: 0.5497
Epoch 10/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7311 - loss: 0.5474 - val_accuracy: 0.7283 - val_loss: 0.5501
Epoch 11/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7325 - loss: 0.5462 - val_accuracy: 0.7321 - val_loss: 0.5474
Epoch 12/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.7323 - loss: 0.5455 - val_accuracy: 0.7311 - val_loss: 0.5470
Epoch 13/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7337 - loss: 0.5441 - val_accuracy: 0.7284 - val_loss: 0.5536
Epoch 14/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.7335 - loss: 0.5443 - val_accuracy: 0.7311 - val_loss: 0.5470
Epoch 15/20
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.7342 - loss: 0.5432 - val_accuracy: 0.7321 - val_loss: 0.5459
```


Accuracy Curve
Train
Val
0.74
0.72
0.70
0.68
0.66
0.64
0
5
10
15
Loss Curve
Train
Val
0.64
0.62
0.60
0.58
0.56
0.54
0
5
10
15


Confusion Matrix
5385
1619
2110
4886
0
1
0
1
5000
4500
4000
3500
3000
2500
2000




```
              precision    recall  f1-score   support

           0       0.72      0.77      0.74      7004
           1       0.75      0.70      0.72      6996

    accuracy                           0.73     14000
   macro avg       0.73      0.73      0.73     14000
weighted avg       0.73      0.73      0.73     14000
```

Conclusion

The Recurrent Neural Network (RNN) with LSTM architecture was successfully implemented on the cardiovascular dataset. Although LSTM is primarily used for sequential data, it was adapted by reshaping tabular data into sequence format. The model demonstrated good predictive

performance and highlighted the ability of deep learning techniques to analyze complex healthcare datasets.